

Digital Differential Analyzer

DDA 알고리즘은 다음과 같은 단계로 수행됩니다.

1. 그리고자 하는 선분의 기울기를 계산 ($m = \frac{y_{end}-y_{start}}{x_{end}-x_{start}}$)
2. 기울기가 양수이고 $m \leq 1$ 일 경우 x축 방향으로 1씩 증가하면서 y값을 계산

$$x_{k+1} = x_k + 1$$

$$y_{k+1} = y_k + m$$

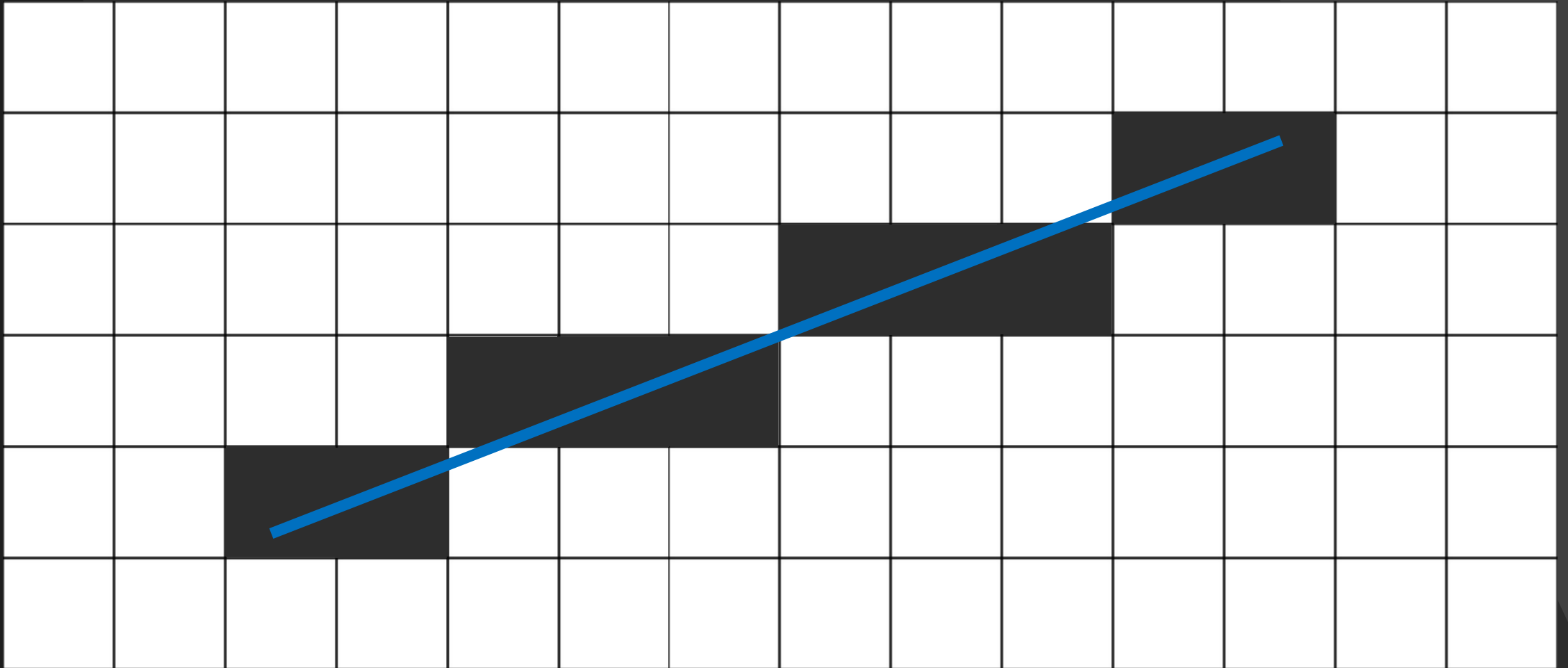
3. $1 < m$ 일 경우 y축 방향으로 1씩 증가하면서 x값을 계산

$$x_{k+1} = x_k + \frac{1}{m}$$

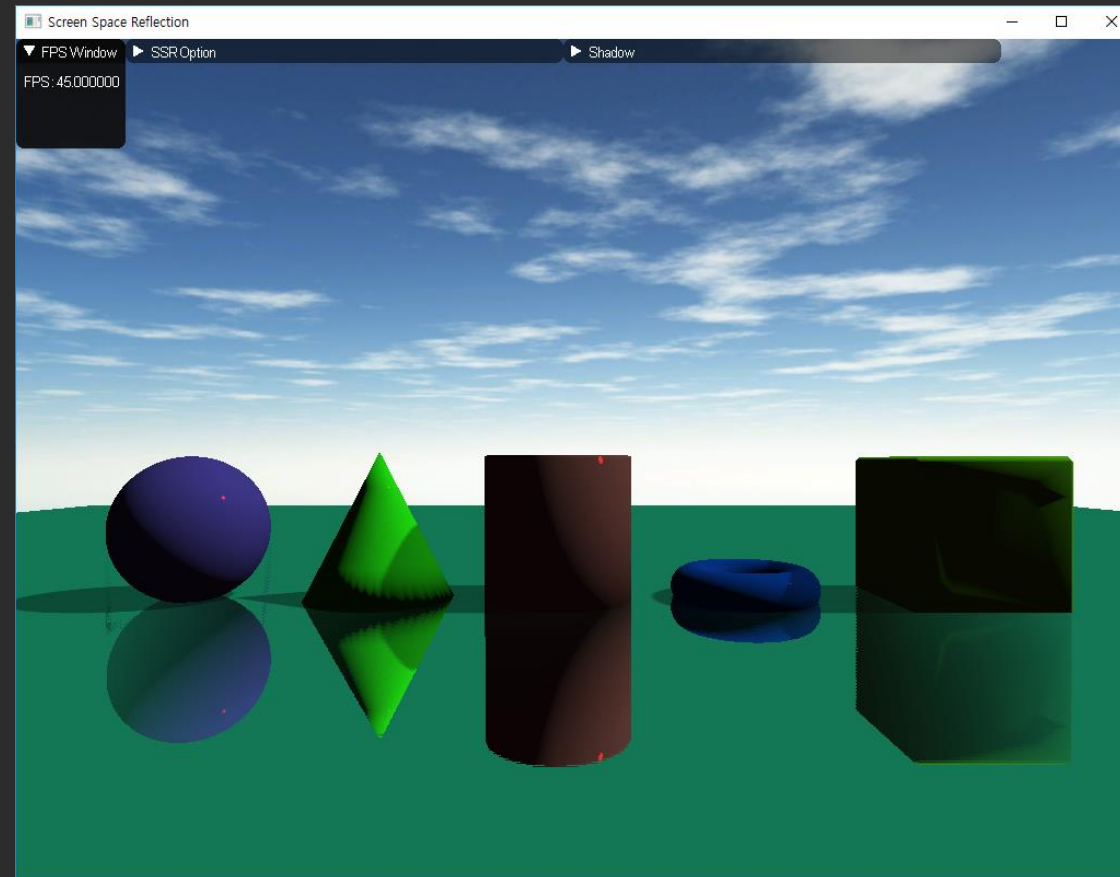
$$y_{k+1} = y_k + 1$$

4. 기울기가 음수 일 경우에도 픽셀 위치를 계산하기 위해 유사한 계산이 이뤄짐

Digital Differential Analyzer



Screen Space Reflection Using Digital Differential Analyzer



Screen Space Reflection Using Digital Differential Analyzer

```
float4 main( PS_INPUT input ) : SV_TARGET
```

```
{
```

```
float3 incidentVec = normalize( input.viewPos );  
float3 viewNormal = normalize( input.viewNormal );  
  
float3 reflectVec = reflect( incidentVec, viewNormal );  
reflectVec = normalize( reflectVec );    반사 벡터 계산
```

```
float3 hitPixel_alpha;  
bool isHit = TraceScreenSpaceRay( reflectVec, input.viewPos, hitPixel_alpha );    충돌 여부 및 충돌한 픽셀 좌표를 계산
```

```
float3 color = framebufferTex.SampleLevel( baseSampler, hitPixel_alpha.xy, 0 ).rgb;
```

```
return float4( color, hitPixel_alpha.z ) * isHit;
```

```
}
```

```
bool TraceScreenSpaceRay( float3 dir, float3 viewPos, out float3 hitPixel_alpha )
```

```
{
```

```
float rayLength = ( viewPos.z + dir.z * g_maxRayLength ) < g_nearPlaneDist ? ( g_nearPlaneDist - viewPos.z ) / dir.z : g_maxRayLength;
```

```
hitPixel_alpha = float3( -1, -1, 1 );
```

```
float3 rayEnd = viewPos + dir * rayLength;
```

```
float4 ssRayBegin = mul( float4( viewPos, 1.0 ), g_projectionMatrix );  
float4 ssRayEnd = mul( float4( rayEnd, 1.0 ), g_projectionMatrix );
```

```
float k0 = 1 / ssRayBegin.w;
```

```
float k1 = 1 / ssRayEnd.w;
```

화면 공간으로 광선의 시작점, 끝점을 변환

Screen Space Reflection Using Digital Differential Analyzer

```
float2 P0 = ssRayBegin.xy * k0;  
float2 P1 = ssRayEnd.xy * k1;  
  
P0 = ( P0 * 0.5 + 0.5 ) * g_targetSize;  
P1 = ( P1 * 0.5 + 0.5 ) * g_targetSize;
```

화면 공간으로 광선의 시작점, 끝점을 변환

```
P1 += ( dot( P1 - P0, P1 - P0 ) < 0.0001 ) ? float2( 0.01, 0.01 ) : float2( 0, 0 );  
float2 delta = P1 - P0;
```

기울기 계산 후 DDA delta 값을 계산

```
bool permute = false;  
if ( abs( delta.x ) < abs( delta.y ) )  
{  
    permute = true;  
    delta = delta.yx;  
    P0 = P0.yx;  
    P1 = P1.yx;  
}
```

```
float stepDir = sign( delta.x );  
float invdx = stepDir / delta.x;
```

```
float stepCount = 0;  
float rayZ = viewPos.z;  
float sceneZMax = viewPos.z + 100.f;
```

```
float end = stepDir * P1.x;
```

```
float3 Pk = float4( P0, k0 );  
float3 dPk = float4( float2( stepDir, delta.y * invdx ), ( k1 - k0 ) * invdx );
```

```
dPk *= g_rayStepScale;
```

```
float thickness = g_maxThickness;
```

$$1 < \frac{y_{end} - y_{start}}{x_{end} - x_{start}} \\ = x_{end} - x_{start} < y_{end} - y_{start}$$

Screen Space Reflection Using Digital Differential Analyzer

```
[loop]
for ( ;
    ( ( Pk.x * stepDir ) <= end ) && // 광선이 끝까지 진행하기 전까지
    ( stepCount < g_maxRayStep ) && // 최대 Ray Marching 단계에 도달하기 전까지
    ( ( rayZ < sceneZMax ) ||
    ( ( rayZ - thickness ) > sceneZMax ) ) && // 물체에 부딪히기 전까지
    ( sceneZMax != 0.0 );
    Pk += dPk,
    stepCount += 1 ) // 화면 공간에서 광선을 진행
{
    hitPixel_alpha.xy = permute ? Pk.yx : Pk.xy;
    hitPixel_alpha.xy *= g_invTargetSize;
    hitPixel_alpha.y = 1 - hitPixel_alpha.y;

    rayZ = 1 / Pk.w;
    sceneZMax = depthbufferTex.SampleLevel( baseSampler, hitPixel_alpha.xy, 0 ).x;
    thickness = backfaceDepthBufferTex.SampleLevel( baseSampler, hitPixel_alpha.xy, 0 ).x * g_FarPlaneDist;
    sceneZMax *= g_FarPlaneDist;
    thickness -= sceneZMax;
    sceneZMax += g_depthbias;
}
```

Ray Marching

```
float edgeFade = 1.f - pow( length( hitPixel_alpha.xy - 0.5 ) * 2.f, 3.0f );
hitPixel_alpha.z = edgeFade;
return ( rayZ >= sceneZMax ) && ( rayZ - thickness <= sceneZMax );
```

```
}
```